DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK APPLICATION DEVELOPMENT

#SKILL - 4 ❾ Spring Dependency Injection – Constructor & Setter Injection

Prerequisites:

- **Basic knowledge of Java**
- **General Idea on Spring Framework**
- **Basic Idea on DI & IoC**

A training institute wants to automate the management of student information in a Spring application. The system should inject details such as **studentId**, **name**, **course**, and **academic year** into objects without manually assigning values. This Skill demonstrates **Constructor Injection** and **Setter Injection** using both **XML** and **Annotation** configurations.

Tasks:

- Create a Java class named **Student** with fields: studentId, name, course, year.

1. Create a **Constructor** that accepts all fields and assigns values.
2. Create **Setter methods** for at least two fields (example: course, year).
3. Configure the Student bean using:
   a. **XML Configuration** - The student should take:
      - a **POJO class**
      - an **XML configuration file**
      - a **main class** to load the container

   b. **Annotation Configuration** - The student should take:
      - a **POJO class**
      - a **Java class with necessary annotations**
      - a **main class** to load the container 4. Create a Spring configuration file (XML or Java-based).

5. Write a main class to load the Spring IoC container.
6. Retrieve the Student bean and print the injected values.
7. **Push the Spring (core) project to GitHub.**

MainApp.java

```
package com.klu.main;

import com.klu.model.CourseRegistration; import
org.springframework.context.ApplicationContext; import
org.springframework.context.support.ClassPathXmlApplicationContext;

public class MainApp {     public static
void main(String[] args) {

      ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

      CourseRegistration cr =
            (CourseRegistration) context.getBean("courseReg");

      cr.display();
   }
}
```

CourseRegistration.java

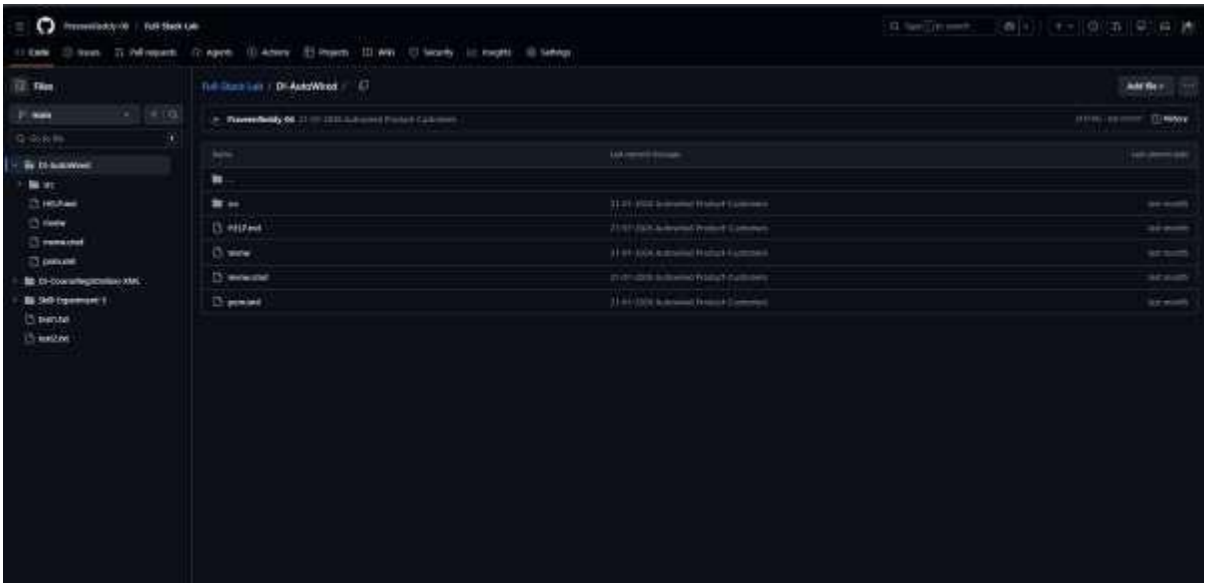
```
package com.klu.model;



public class CourseRegistration {
private int rollNo;     private
String studentName;     private
String courseName;
    private int semester;

    public CourseRegistration(int r, String sn) {
        this.rollNo = r;
        this.studentName = sn;
    }

    public void setCourseName(String cn) {
        this.courseName = cn;
    }

    public void setSemester(int s) {
        this.semester = s;
    }

    public void display() {
        System.out.println("RollNO: " + rollNo);
        System.out.println("Name: " + studentName);
        System.out.println("Course: " + courseName);
        System.out.println("Semester: " + semester);
    }
}
```

ApplicationContext.xml

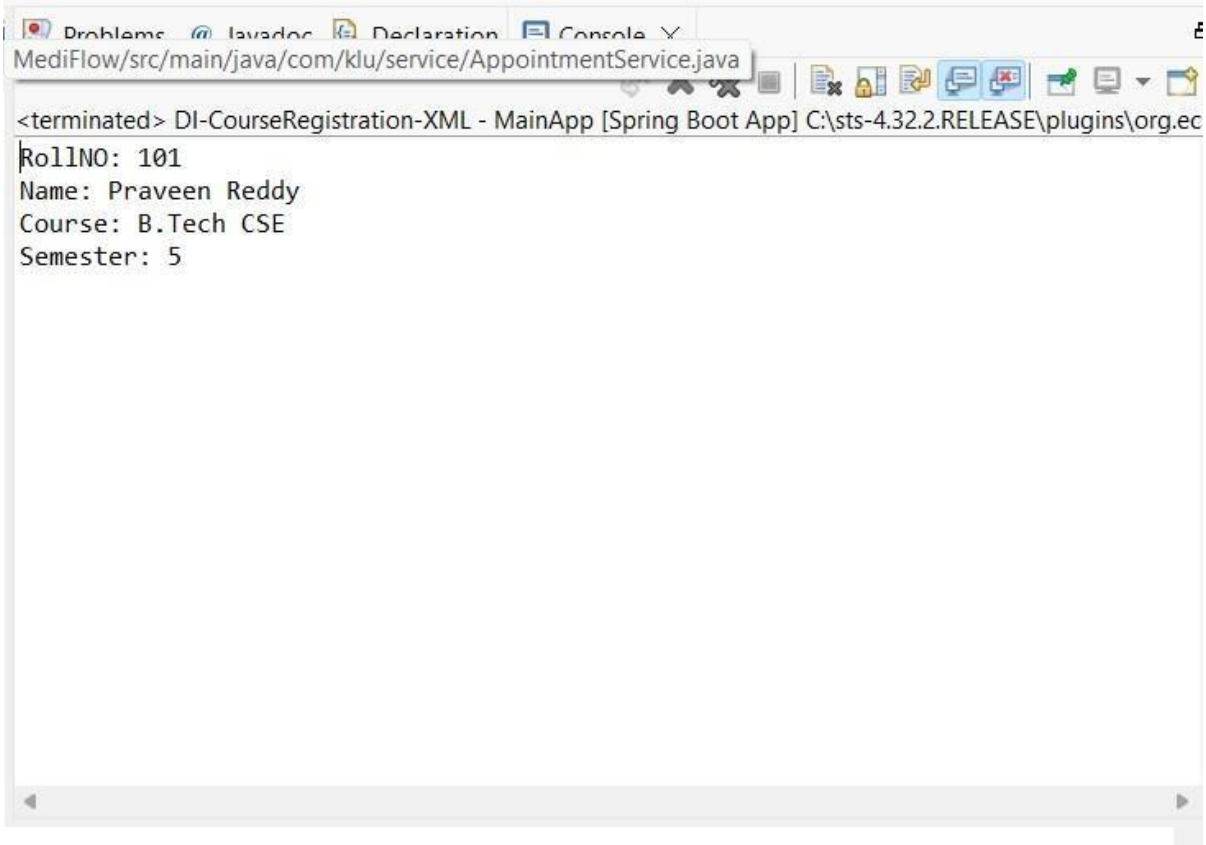
```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"         xsi:schemaLocation="
        http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd">


    <bean id="courseReg" class="com.klu.model.CourseRegistration">


        <constructor-arg value="101"/>
        <constructor-arg value="Praveen Reddy"/>
```

```
        <property name="courseName" value="B.Tech CSE"/>
        <property name="semester" value="5"/>

    </bean>

</beans>
```

GitHub



OUTPUT



GitHub Link : https://github.com/Anil1843/fsad-skill

VIVA QUESTIONS:

1. What is Dependency Injection (DI) in Spring?

Dependency Injection is a design pattern where the Spring framework provides required objects (dependencies) to a class instead of the class creating them itself. This promotes loose coupling and easier maintenance.

2. What is the difference between Constructor DI and Setter DI?

Constructor DI	**Setter DI**
Dependencies are injected through the constructor.	Dependencies are injected using setter methods.
Ensures required dependencies are provided at object creation.	Allows optional dependencies.
Makes the object immutable (preferred for mandatory fields).	Fields can be changed later.

3. What is the purpose of the IoC container?

The Inversion of Control (IoC) container manages the lifecycle of Spring objects (beans). It creates objects, injects dependencies, configures them, and handles their destruction.

4. Why is DI preferred over manual object creation?

1. tight coupling between classes
2. Improves testability
3. Makes code easier to maintain
4. Enhances reusability
5. Centralizes configuration

5. Can Constructor Injection and Setter Injection be used together?

Yes. Both can be used in the same class — typically constructor injection for mandatory dependencies and setter injection for optional ones.

(For Evaluator's use only)

Comment of the Evaluator (If Any)

Evaluator's Observation

Marks Secured: _____out of _____

EMP ID & Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation: